PROJECT PROPOSAL

WAREHOUSE EQUIPMENT MONITORING AND INVENTORY OPTIMIZATION USING CLOUD-BASED DIGITAL TWIN ARCHITECTURE

QUANTUM CLUSTERS

Contents

1 Introduction

1.1 Industry Background & Current Challenges

The supply chain and logistics industry plays a critical role in the global economy by enabling the movement of goods from manufacturers to end customers. Within this ecosystem, warehouses serve as essential hubs for inventory storage, order processing, and distribution coordination. As customer expectations for speed, accuracy, and visibility increase, warehouse performance has become a major determinant of supply chain efficiency and competitiveness.

To improve operational control, many organizations have adopted tools such as Warehouse Management Systems (WMS), barcode and RFID scanning, and basic automation technologies. These systems provide structured data and help streamline specific tasks; however, they often operate in isolation and provide only periodic or static updates. As a result, stakeholders lack a complete, real-time view of warehouse operations, making it difficult to anticipate problems or optimize performance.

1.2 Shift Toward Intelligent Solutions and Project Focus

In response to growing complexity and volatility in supply chain operations, the industry is shifting toward data-driven and intelligent solutions. Technologies such as IoT, cloud computing, machine learning, and simulation are being integrated to improve visibility and decision-making. Among these innovations, digital twin technology has emerged as a transformative approach that creates a dynamic virtual replica of physical warehouse operations. By continuously integrating live data from equipment, inventory systems, and sensors, a digital twin enables real-time monitoring, predictive analysis, and scenario simulation.

This project focuses on designing an AWS-based digital twin solution for warehouse operations. The goal is to create a cloud-powered virtual representation of warehouse assets and processes that reflects real-time conditions, support predictive maintenance, enhance inventory visibility, and enables operational optimization. By leveraging AWS services for data ingestion, modeling, analytics, and visualization, the proposed solution aims to deliver a scalable and intelligent platform that improves efficiency, accuracy, and decision-making across the warehouse environment.

2 Business Requirements

2.1 Industry Challenges and Stakeholder Requirements

Although warehouses have adopted systems such as WMS, RFID, and automation, research indicates that these technologies often function in isolation and lack real-time visibility across equipment, inventory, and workflows. This results in several ongoing operational issues. Inventory data may not accurately reflect physical stock levels, leading to stockouts or excess inventory. Equipment failures occur without warning, causing downtime and delays. Inefficient warehouse layouts and poorly optimized workflows increase travel time and slow order fulfillment. Additionally, decision-making remains largely reactive, as stakeholders do not have synchronized or predictive insights into warehouse performance. These persistent challenges affect operational efficiency, customer satisfaction, and cost control.

A digital twin solution must address the needs of multiple stakeholders involved in daily warehouse operations. Warehouse managers require real-time visibility into inventory levels, equipment status, and workflow performance to plan and coordinate resources effectively. Maintenance technicians need predictive insights and alerts to prevent equipment failures and reduce downtime. Inventory planners rely on accurate, up-to-date stock information to optimize replenishment and avoid shortages or overstock situations. Operations supervisors need tools to monitor productivity, identify bottlenecks, and improve task allocation. Executive leadership depends on high-level performance metrics and cost transparency to evaluate efficiency and make strategic decisions. By delivering an integrated, data-driven view of warehouse operations, a digital twin can support each of these stakeholder groups.

2.2 Business Objectives

The primary business objectives of implementing a digital twin solution on AWS are:

- Increase equipment uptime through predictive maintenance and early anomaly detection.
- Improve inventory accuracy by synchronizing physical stock levels with system data in real time.
- Optimize warehouse layout and workflows to reduce travel time and accelerate order fulfillment.
- Enable data-driven decision-making with centralized dashboards, performance metrics, and operational transparency.
- Enhance operational efficiency and productivity by reducing manual intervention and improving resource utilization.
- Lower long-term operational costs by minimizing downtime, labor inefficiencies, and inventory waste.

2.3 Industry-Specific Pros, Cons, and Expected Outcomes

A digital twin solution offers significant advantages for warehouse operations. It provides real-time visibility across assets and processes, supports predictive intelligence for maintenance and inventory management, and enables simulation to optimize workflows before implementation. These capabilities lead to higher accuracy, faster decision-making, and continuous operational improvement. However, industry research also identifies certain challenges. Implementing a digital twin may require integration with legacy systems, high-quality data collection, initial setup cost, and staff training to ensure adoption. Despite these considerations, the expected outcomes of a digital twin solution include improved equipment uptime, higher inventory accuracy, shorter fulfillment times, greater operational transparency, and reduced overall costs. When supported by scalable AWS cloud services for data ingestion, modeling, analytics, and visualization, digital twin technology becomes a strategic and sustainable approach to warehouse modernization.

3 User Requirements and Use Case Definition

3.1 Use Case Overview

Our system uses AWS cloud technology to build a digital twin of a warehouse, a virtual version that mirrors everything happening in the real one. Using IoT sensors and smart devices, the system collects real-time data about machines, equipment, and inventory. This information is sent to AWS, where it is processed and displayed on a live dashboard.
The warehouse manager can use this system to monitor machine performance, check stock levels, and predict maintenance needs before breakdowns occur. It can also suggest better ways to organize goods, improve picking routes, and reduce delays in order processing. Overall, the system helps make warehouse operations smarter, faster, and more reliable.

3.2 Users

- **Warehouse Manager:** Oversees warehouse operations and ensures everything runs smoothly.
- **Maintenance Technician:** Responsible for keeping machines and equipment working properly.
- **Inventory Controller:** Monitors stock levels and ensures accuracy between physical and system data.
- **Operations Analyst:** Analyzes warehouse data to improve efficiency and reduce costs.

3.3 User Requirements

- The system should provide real-time tracking of inventory and equipment conditions using IoT sensors.
- The system should send alerts and notifications when machines show signs of wear or need maintenance.
- The system should allow users to view a digital model (digital twin) of the warehouse to track activities live.
- The system should help predict equipment failures using data analytics and machine learning on AWS.
- The system should offer reports and insights to improve warehouse layout, reduce downtime, and speed up operations.

4 Solution Requirements

4.1 Functional Requirements

Real-Time Equipment Data Ingestion
In the warehouse, every piece of equipment, such as forklifts, robotic arms, generates data constantly. This data includes multiple things: Temperature (to detect overheating), energy consumption, vibration level, and so on. This data can be collected by using sensors, local gateways (AWS IoT Greengrass).

Data Processing and Event Handling
Once data is received by AWS, it should be processed instantly using AWS Lambda (Helps for event driven automation) and AWS Kinesis (Uses for real time data stream)

This service cleans and analyzes incoming data, removes unnecessary information, and readings. This would also monitor the anomalies for example, If a machine temperature suddenly upsurges. This would help to detect the problem at the moment they start, not after the equipment fails.

Centralized Data Storage and Structuring
All the data should be saved securely in a centralized data storage (Cloud Storage) for the future.

AWS IoT SiteWise or Dynamo DB can be used for organized and structured data. This would help to process the data with other AWS tools.

Digital-Twin Synchronization
The system should create a virtual copy of the warehouse layout or its equipment using AWS IoT TwinMaker. This digital twin updated automatically based on realtime IoT data, which shows which machine is working, and how inventory is moving. It provides a live status of each machine and inventory zone.

Predictive Maintenance and Alerts
Using Amazon SageMaker (Machine Learning), the system should learn from past trends and data to predict equipment failures and trigger alerts through Amazon SNS or Notification.

Inventory Tracking and Optimization
The system should be capable of tracking the inventory movement and automatically update digital twin inventory levels, suggesting optimal storage zone and reordering action based on utilization.

Dashboard Visualization and Reporting
User must be able to view key warehouse KPI's (Equipment, Temperature and Inventory Status) in an attractive Amazon QuickSight dashboard.

4.2 Non-Functional Requirements (System Qualities)

These define how well the system performs, and its quality attributes.

Scalability
The system should automatically handle growth to process data from multiple warehouses and thousands of IoT devices without any lagging. AWS services like IoT Core, Lambda and S3 automatically scale up depending on how much data is coming in.

Reliability and Availability
The system should be capable of achieving 99.9% uptime. Ideally, this can be achieved by using AWS Multi Availability Zones, in case one region fails; another will overtake critical services like IoT Core & Databases.

Security
Security ensures that all the data is encrypted in transit and at rest using tools like AWS KMS, or IAM policies.

Performance/Latency
System should process and update data in real time and should instantly appear in the dashboard. AWS Lambda and Kinesis are used for event-based data processing with minimal delays.

Maintainability and Observability
The system should support automated monitoring through AWS CloudWatch and allow easy deployment updates via CloudFormation/CDK, ensuring minimal downtime during maintenance.

5 AWS Architecture Design

5.1 Architecture Diagram and Communication Flow

The system architecture comprises seven layers that together enable real-time data ingestion, processing, analytics, and visualization as shown in Figure 1. This proposal presents the conceptual version of AWS architecture. A more detailed model showing service-level configurations and data paths will be developed in the final report.

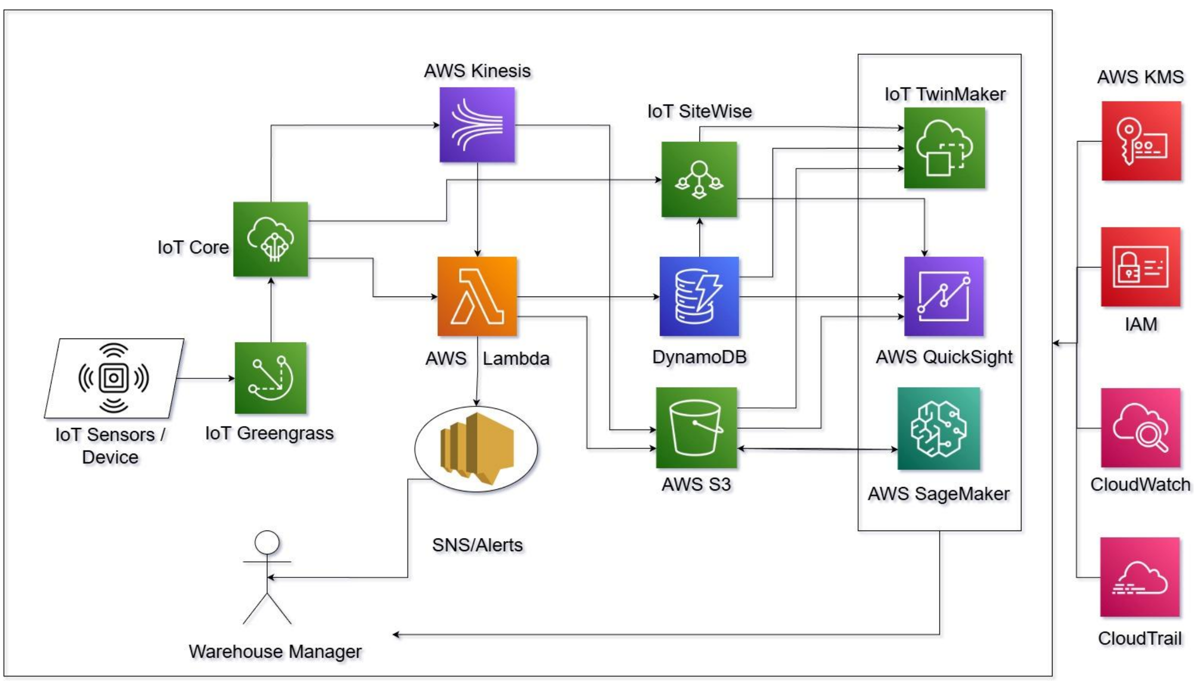

Figure 1 AWS Based Warehouse Digital Twin Architecture

Edge Layer (Warehouse IoT Environment)
IoT sensors installed on forklifts, conveyor belts, and storage racks collect real-time data such as temperature, vibration, energy usage, and stock levels. AWS IoT Greengrass runs on local gateways to perform basic preprocessing, filtering, and buffering of sensor data and to support limited offline

operation when connectivity is unstable. AWS IoT Core provides a secure channel that connects these devices and gateways to the AWS cloud.

Data Ingestion and Stream Processing
AWS IoT Core acts as the primary entry point for all IoT telemetry from the warehouse. From there, Amazon Kinesis Data Streams or Kinesis Data Firehose is used to stream this continuous data for further processing. AWS Lambda functions are triggered from these streams to transform and clean the incoming messages, discard invalid or duplicate readings, and detect simple anomalies such as sudden spikes in equipment temperature or vibration.

Data Storage and Structuring
After processing, the data is stored in multiple services depending on its usage. Amazon DynamoDB holds structured metadata, equipment logs, and the latest status indicators that need fast lookup. Amazon S3 stores raw and historical IoT data, which is used later for analytics, reporting, and machine learning training. AWS IoT SiteWise is used to model and organize industrial equipment data, such as asset hierarchies and time-series measurements, so that it can be easily consumed by other AWS services.

Digital Twin Creation
AWS IoT TwinMaker is used to build and maintain the digital twin of the warehouse. It combines 3D scene data, equipment models, and live telemetry to create a virtual representation of the physical environment. Amazon S3 and AWS IoT SiteWise feed 3D assets and time-series metrics into TwinMaker, allowing the digital twin to update in near real time as conditions in the warehouse change. Users can view this model to see the current state of machines, inventory zones, and material flows.

Analytics and Machine Learning
Amazon SageMaker trains and hosts machine learning models that use historical and real-time data to predict equipment failures, identify abnormal behavior, and support layout or process optimization. When a model detects a potential issue, Amazon Simple Notification Service (SNS) sends alerts to warehouse managers and maintenance technicians through email, SMS, or integrated applications.

Visualization and Reporting
Amazon QuickSight provides interactive dashboards that display key warehouse metrics such as equipment health, temperature trends, throughput, and inventory accuracy. These dashboards are backed by data from DynamoDB, S3, and IoT SiteWise. AWS CloudWatch collects logs and performance metrics from the underlying services, giving operators a single place to monitor the health and performance of the architecture itself.

Security and Governance
Security and governance are enforced using native AWS services. AWS Identity and Access Management (IAM) controls which users, roles, and devices can access specific resources or

perform particular actions. AWS Key Management Service (KMS) manages encryption keys so that data remains protected both in transit and at rest. AWS CloudTrail records API activity across the environment, providing an audit trail that can be used for compliance checks and operational investigations.

In this architecture, IoT sensors first collect real-time equipment and inventory data, which is preprocessed by AWS IoT Greengrass and sent securely to AWS IoT Core. The data then flows through Amazon Kinesis, where AWS Lambda functions filter and enrich the stream before writing it to Amazon S3, Amazon DynamoDB, and AWS IoT SiteWise. AWS IoT TwinMaker uses this data to keep the warehouse digital twin in sync with physical operations. Amazon SageMaker analyzes historical patterns to generate predictive insights and triggers alerts through Amazon SNS when risks are detected. Finally, Amazon QuickSight and AWS CloudWatch provide visualization and monitoring capabilities, giving stakeholders a clear and timely view of warehouse performance across all layers of the system.

5.2 Justification of Each AWS Tool

The selected AWS services were chosen to build a secure, scalable, and efficient architecture that supports continuous data flow, predictive analytics, and visualization for the warehouse digital twin. Each component has a clear role and was evaluated against other available options within AWS.

AWS IOT Core
It acts as the main connectivity layer for all sensors and devices in the warehouse. It provides secure communication between IoT gateways and the AWS cloud using standard messaging protocols and managed authentication. This service was chosen instead of custom API based ingestion or message brokers because it directly supports large volumes of IoT messages with built in security and device management, which matches the needs of an active warehouse environment.

AWS IoT Greengrass
It runs on local gateways in the warehouse and handles basic data filtering, aggregation, and buffering when internet connectivity is unstable. It ensures that important readings are not lost and reduces the amount of raw data sent to the cloud. It was selected over custom edge scripts because it integrates natively with IoT Core and Lambda, supports offline operation, and reduces the development effort required to manage logic at the edge.

Amazon Kinesis Data Streams
Kinesis Data Streams is used to stream continuous IoT data from IoT Core into the processing layer. It provides durable, ordered, and scalable handling of events as they arrive from the warehouse. It was chosen instead of Amazon MSK or direct writes to databases because it is better suited for high volume telemetry with low delay and ties in naturally with Lambda for event driven analytics.

AWS Lambda
Lambda processes incoming data events by cleaning records, transforming formats, and detecting

basic anomalies. It scales automatically with the volume of messages and does not require any server management. Lambda was preferred over EC2 based processing since it removes the need to maintain infrastructure, fits short running IoT processing tasks, and reduces operational overhead for this project.

Amazon DynamoDB

DynamoDB stores structured information such as equipment status, device metadata, and recent logs that are needed quickly by dashboards and services. It offers fast read and write performance even during load spikes.

It was selected instead of Amazon RDS or Aurora because the data model is key value based, the workload is highly variable, and DynamoDB handles this pattern without manual capacity planning.

Amazon S3

S3 is used to store raw sensor data, historical logs, and training datasets for machine learning models. It provides durable storage and supports lifecycle rules for moving older data to lower cost tiers. S3 was chosen instead of EFS or EBS because the project needs to store large volumes of append only IoT data in a cost efficient way, without attaching storage to specific servers.

AWS IoT SiteWise

SiteWise organizes equipment data into asset models, hierarchies, and time series metrics. This makes it easier to understand relationships between machines, locations, and performance indicators. It was selected because it provides a structured industrial data model out of the box, which reduces the effort required to manually manage asset relationships for the digital twin.

AWS IoT TwinMaker

TwinMaker is responsible for building and updating the digital twin of the warehouse. It combines 3D scenes, asset models, and live data so users can see the current operational state visually. It was preferred over building a custom visualization layer because it already connects with SiteWise and S3, and it is designed specifically for digital twin scenarios that match the goals of this project.

Amazon SageMaker

SageMaker is used to train and deploy machine learning models that predict equipment failures and support optimization decisions. It manages the full model lifecycle from training to hosting. SageMaker was chosen instead of running frameworks on EC2 because it simplifies experimentation, reduces setup time, and provides managed endpoints that integrate easily with Lambda and other AWS services.

Amazon SNS

SNS sends alerts to warehouse managers and technicians when a model or rule identifies a risk or anomaly. It can deliver messages through multiple channels such as email or text. SNS was selected instead of Amazon SQS or external messaging tools because the project needs immediate push notifications to people, not queue based processing between systems.

Amazon QuickSight
QuickSight provides dashboards that display key performance indicators such as uptime, inventory accuracy, and throughput. It connects directly to S3, DynamoDB, and other AWS data sources. QuickSight was chosen instead of self-hosted BI tools because it does not require separate servers, is simple to share with users, and fits well with a cloud native analytics stack.

AWS CloudWatch
CloudWatch monitors logs, metrics, and alarms from all components in the architecture. It allows the team to track performance, errors, and resource usage from a single place. It was selected because it is integrated automatically with almost every AWS service, which removes the need to maintain a separate monitoring platform.

AWS IAM and AWS KMS
AWS Identity and Access Management (IAM) controls access for users, roles, and devices, while AWS Key Management Service (KMS) manages encryption keys to protect data. Together, they secure the environment. They were chosen instead of custom access control and encryption logic because they are standard across AWS, support fine grained permissions, and meet security expectations for operational data.

AWS CloudTrail
CloudTrail records API calls and configuration changes across the account. It keeps an audit log of administrative actions. CloudTrail was selected because it provides built-in traceability, which is important for investigating changes and demonstrating governance over the digital twin environment.

These services together create a tightly integrated AWS architecture that supports continuous data ingestion, warehouse visualization, predictive insights, and secure operations for the digital twin solution.

5.3 Alternative Cloud Tools

While the final architecture is designed entirely within the AWS ecosystem, several alternative tools were evaluated during the planning stage. Within AWS, services such as Amazon MSK, IoT Analytics, and Firehose were reviewed for data ingestion and streaming. Although these tools offer similar capabilities, they require additional configuration or deliver less control over throughput and event handling. For data storage, options like EFS and EBS were considered, but they are optimized for block or file level workloads rather than continuous sensor data. Amazon S3 was therefore retained for its durability, flexibility, and cost suitability for large volumes of IoT information. In the analytics layer, third-party visualization tools such as Tableau and Power BI were explored; however, Amazon QuickSight provided a simpler integration path with other AWS components and required no external hosting or connectors.

Beyond the AWS suite, the team reviewed broader industry alternatives such as Azure IoT Hub and Google Cloud IoT Core. While these platforms provide comparable device connectivity and analytics capabilities, the proposed design benefits from AWS's unified integration between IoT Core, Kinesis, Lambda, and TwinMaker. This integration reduces deployment complexity and ensures smooth data movement from sensors to visualization. Considering implementation effort, scalability, and cost, the selected AWS services deliver the most appropriate combination of performance and manageability for a mid-scale warehouse digital twin solution.

5.4 Cost Estimation and Scalability Analysis

The expected performance outcomes and operational impacts of the proposed AWS-based digital twin system are summarized in Table 1.

Table 1 Performance Outcomes and Operational Impact of the Proposed AWS-Based Digital Twin Solution

Category	Performance Outcome	Impact
Energy Efficiency	18-22% reduction in total warehouse energy consumption	Achieved by predictive maintenance minimizing equipment idle time and optimizing energy usage of forklifts, conveyors, and lighting systems.
Equipment Downtime	28-35% reduction in unplanned maintenance-related downtime	Enabled through real-time anomaly detection and predictive analytics using Amazon SageMaker.
Inventory Accuracy	95-99% synchronization between physical stock and digital records	Real-time IoT tracking and automated reconciliation through AWS IoT SiteWise and DynamoDB.
Order Fulfillment Speed	25-30% faster average processing time per order	Optimized warehouse routing, workflow simulation, and congestion detection using AWS IoT TwinMaker.
Operational Costs	15-18% annual cost reduction	Resulting from minimized waste, enhanced equipment usage, and a decrease in manual interventions.
Process Optimization	32-36 days of cumulative process time saved annually	Achieved via faster inventory movement, predictive restocking, and automated reporting.
Sustainability Impact	8-10% water and resource usage reduction	As a result of improved machine scheduling and decreased idle heat production, the consumption of cooling and cleaning water reduced

A critical aspect of implementing a cloud-based digital twin solution is ensuring cost efficiency and scalability as data volume and warehouse operations expand. Using the AWS Pricing Calculator, we estimated the monthly operational cost of the proposed system based on a mid-sized warehouse environment with approximately 500 IoT sensors transmitting real-time data to the cloud.

5.4.1 Estimated Monthly Cost Breakdown

Table 2 Cost Estimation Breakdown

AWS Service	Usage Details	Estimated Monthly Cost (USD)
AWS IoT Core	500 devices sending data every 10 seconds (~129M messages/month)	$215
Amazon Kinesis Data Streams	10 shards for real-time data streaming and storage	$150
AWS Lambda	5 million function invocations for data transformation and anomaly detection	$25
Amazon DynamoDB	100 GB structured data + 5 million reads/writes	$85
Amazon S3	500 GB data storage + 2 TB retrievals for analytics and visualization	$55
AWS IoT SiteWise	100 assets, 250M data points/month	$135
AWS IoT TwinMaker	1 workspace, 5 scenes, and 20 connected components	$180
Amazon SageMaker	1 notebook instance + 50 training hours + hosted model endpoint	$310
Amazon QuickSight	5 authors and 20 readers accessing dashboards	$70
AWS CloudWatch	10 dashboards and 50 performance alarms	$40
Amazon SNS	100,000 notifications per month for alerts	$5

Total Estimated Monthly Cost: $1,270/month

This projected expense represents consistent usage within standard warehouse operations. Due to architecture's dependence on managed and serverless services. This enables the system to stay cost-effective during times of reduced activity while still accommodating greater demands when warehouse throughput rises.

5.4.2 Cost Optimization Opportunities

The AWS architecture allows several cost-saving mechanisms to be applied dynamically:

- IoT Message Batching and Compression: Lowers the cost of using AWS IoT Core by roughly fifteen percent through a reduction in the frequency of message transmissions.
- S3 Intelligent Tiering: Automatically transfers older or less often accessed data to more economical storage classes with savings of up to twenty percent on data storage.
- On Demand SageMaker Endpoints: Disabling inference endpoints when they are not in use can lead to a reduction in the cost of monthly machine learning hosting by approximately $100-$150.
- AWS Budgets and Cost Explorer Integration: This practice smooths the ongoing oversight of resource usage, automated notifications about costs, and anticipatory budgeting to ensure financial management.

After applying these optimizations, the expected monthly cost decreases to approximately $850–$950, making the system highly cost-efficient for medium-scale warehouse operations.

Scalability Enablement:
Scalability is a core design principle of the proposed AWS-based system:

- Elastic Compute and Storage: Amazon S3, Lambda, and DynamoDB doesn't require any manual provisioning as it automatically adjusts their capacity in response to data and demand volume.
- Multi-Warehouse Integration: AWS IoT TwinMaker and IoT Core can simultaneously be used to model and coordinate multiple warehouse locations.
- Distributed Architecture: The deployment across multiple availability zones guarantees resilience and ensures system uptime > 99.9%.
- Serverless Framework: This approach decreases maintenance efforts and aids swift growth in connected devices or processing capabilities as business requirements change.

Together, these characteristics guarantee that the digital twin system, as it evolves from a single warehouse configuration to a widespread logistics network solution, remains adaptable, sustainable, and economically viable.

6 Future Work

The proposal has been written with the current challenges of warehouse operations. It also identified the key stakeholders and their needs, as well as defined the business, user, functional, and non-functional requirements for a modern solution. The system is going to be an AWS-based digital twin that will provide real-time visibility, predictive maintenance, and inventory optimization in a single integrated platform. It is a new approach that incorporates the issues of both equipment monitoring and inventory, thus increasing the power to scale, the level of transparency, and the quality of decision-making. The team will work on a more elaborate model that will not only show the various service-level integrations and the data flow across components but will also make the AWS architecture more detailed in the next phase. They will be concentrating on the concepts of the IoT data simulation process, the digital twin configuration in AWS IoT TwinMaker, and the implementation of predictive maintenance through Amazon SageMaker in the final report.

7 References

[1] Ivanov, M., & Dolgui, B. (2024). *A systematic review of digital twin applications in supply chain and logistics. Journal of Manufacturing Systems.*
[2] Amazon Web Services. (2023). *AWS IoT TwinMaker: Enabling digital twins for industrial operations.* AWS Whitepaper.
[3] Amazon Web Services. (2023). *Coca-Cola uses AWS IoT SiteWise to improve real-time data visibility.* AWS Case Study.
[4] DHL. (2022). *Digital twins in logistics: A DHL perspective.* DHL Trend Research.
[5] Leng, S., Jiang, X., & Wang, Y. (2023). *Digital twin-driven warehouse management systems for operational optimization. Applied Sciences, 13*(4), 1–15.
[6] Siemens. (2023). *Optimizing warehouse operations with predictive analytics and digital twin technology.* Siemens Case Study.
[7] Amazon Web Services. (2024). AWS Pricing Calculator. Retrieved from https://calculator.aws